Proposal Document

Phase : 01

Project Overview:

A chatbot system where a user pastes a Job Description (JD), and the system retrieves relevant content from a knowledge base of past proposals to generate a tailored cover letter in PDF format. If the user then requests a full proposal, the system generates a detailed proposal document, also in PDF, matching the company's standard template.

Tech Stack:

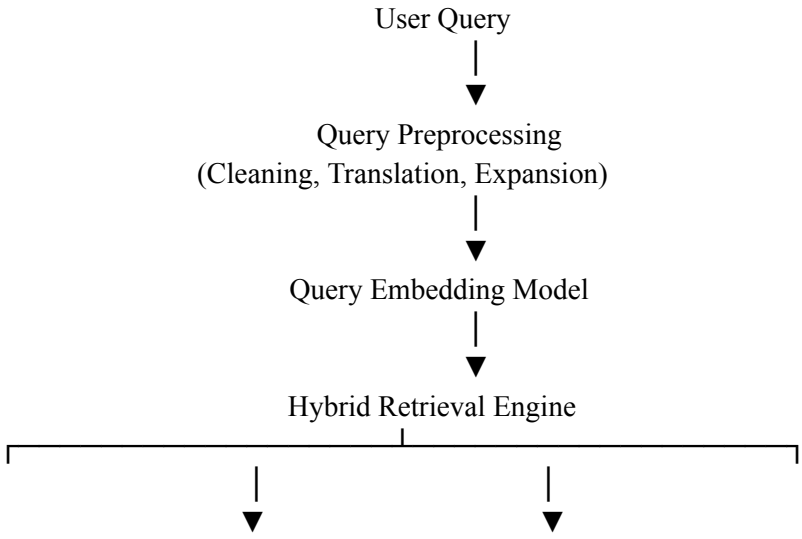
- **PDF Text Extraction:** `pdfplumber/PyMuPDF/Docling` — extracts text from old proposal PDFs
- **LLM (Classification + Generation):** Anthropic Claude API — categorizes proposals & generates new proposal content
- **Chunking:** Custom Python logic (section-based) + `langchain` splitter as fallback — breaks proposals into clean, retrievable sections
- **Embeddings:** `sentence-transformers` (all-MiniLM-L6-v2) — converts text chunks into vectors for semantic search
- **Vector Database:** `Qdrant` — stores and searches proposal chunks with category metadata
- **Backend Framework:** `FastAPI` (Python) — API layer connecting chatbot, retrieval, and PDF generation
- **PDF Generation:** `ReportLab` generates the final proposal in the company's standard format
- **Frontend / Chat Interface:** `React.js` (or `Streamlit` for MVP) — user interface to paste JD and receive proposal
- **Database (metadata/logs):** `PostgreSQL` or `SQLite` (for MVP) — stores proposal metadata, generation history, user sessions

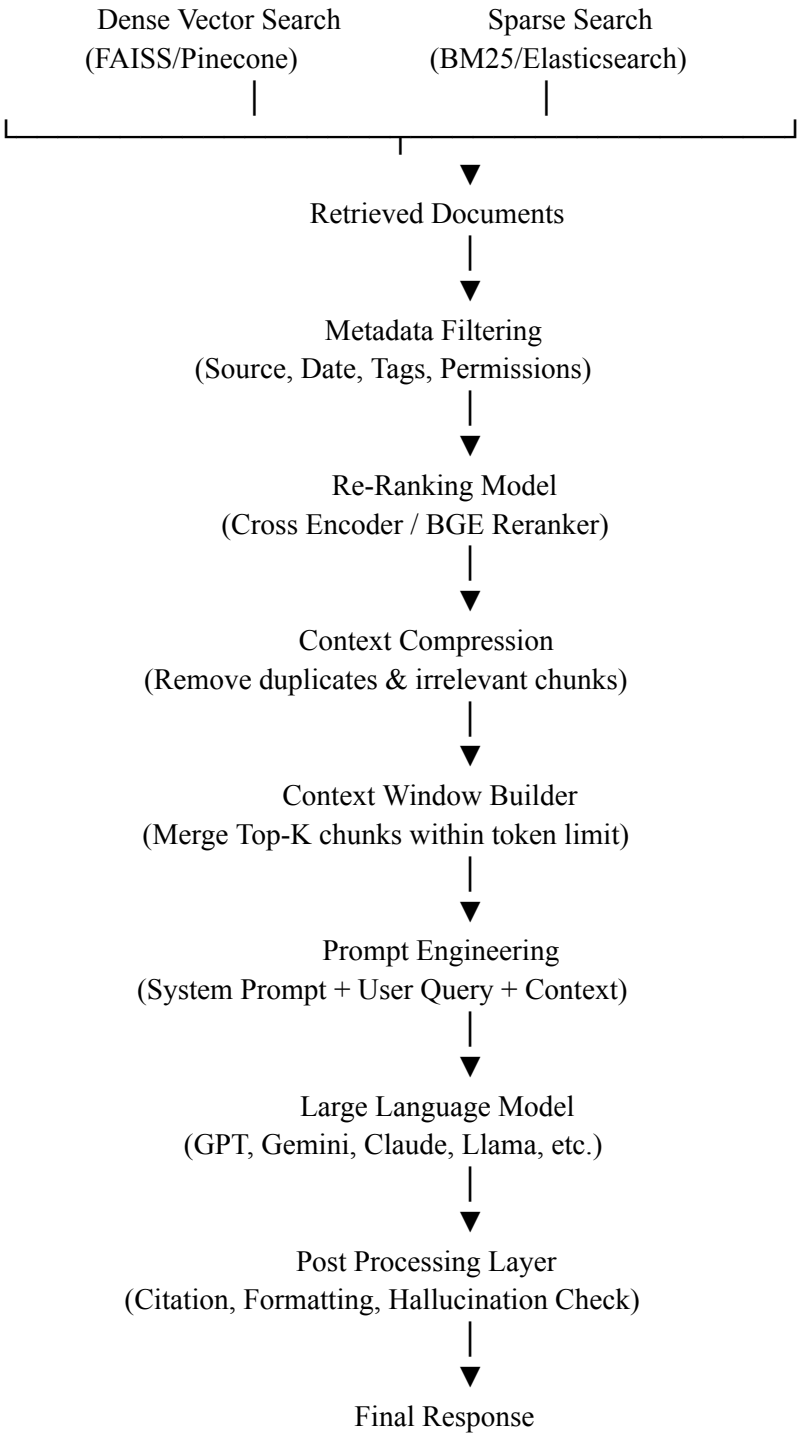
Timeline (Manual Development — No AI-generated code)

- **Phase 1: Setup & Data Collection**
  - Collect all past proposal PDFs
  - Set up project structure and environment
- **Phase 2: PDF Extraction & Categorization**
  - Build PDF text extraction pipeline
  - Build LLM-based categorization script

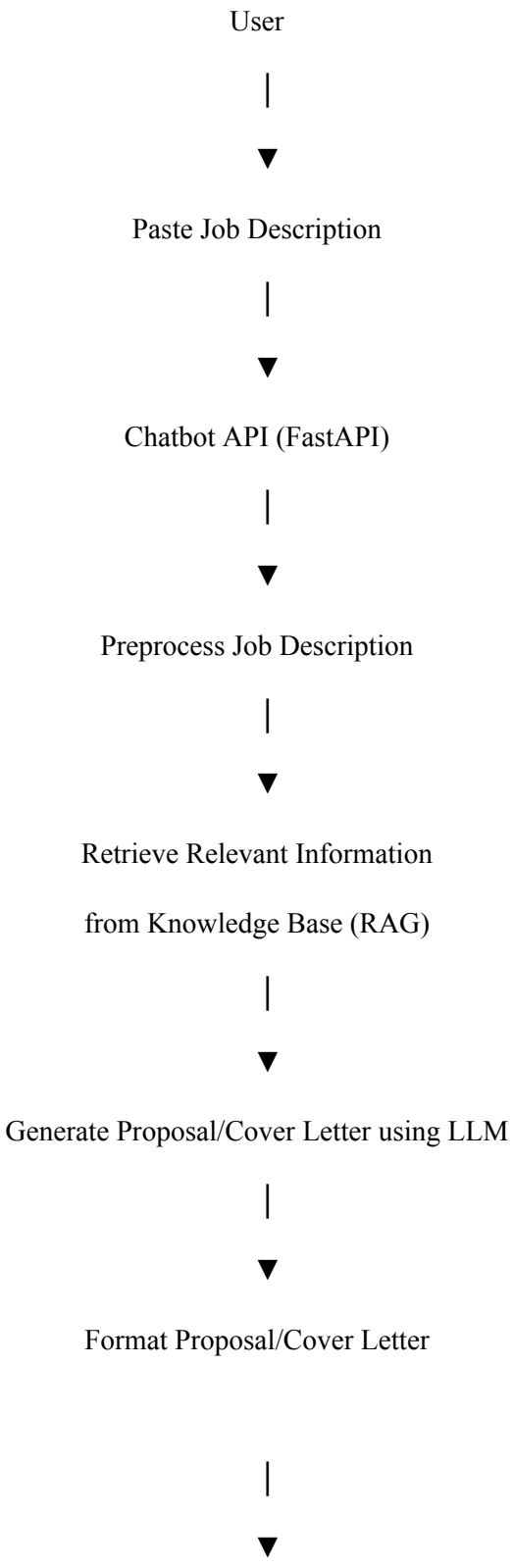
    - Test on sample proposals
- **Phase 3: Chunking Logic**
    - Build section-based chunking
    - Add fixed-size fallback for long sections
    - Validate chunk quality
- **Phase 4: Embeddings & Vector DB**
    - Set up embedding model
    - Integrate Qdrant
    - Store chunks with metadata (category, section, file name)
- **Phase 5: Retrieval Logic**
    - Build retrieval function
    - Implement category-filtered semantic search based on JD input
- **Phase 6: Proposal Generation Logic**
    - Build LLM prompt pipeline
    - Generate structured proposal content (Overview, Scope, Timeline, Pricing, etc.) from retrieved chunks
- **Phase 7: PDF Template & Auto-Fill**
    - Design standard proposal template (HTML/ReportLab)
    - Build logic to auto-fill generated content into template
- **Phase 8: Backend API (FastAPI)**
    - Build endpoints: upload JD, trigger retrieval, trigger generation, return PDF
- **Phase 9: Frontend/Chat Interface**
    - Build UI for pasting JD and downloading generated proposal PDF
- **Phase 10: Testing & QA**
    - End-to-end testing
    - Fix chunking/retrieval issues
    - Validate PDF formatting and edge cases

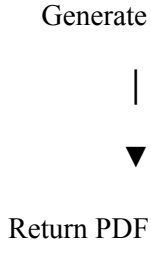
Advance RAG Structure :




Dense Vector Search
(FAISS/Pinecone)
Sparse Search
(BM25/Elasticsearch)
Retrieved Documents
Metadata Filtering
(Source, Date, Tags, Permissions)
Re-Ranking Model
(Cross Encoder / BGE Reranker)
Context Compression
(Remove duplicates & irrelevant chunks)
Context Window Builder
(Merge Top-K chunks within token limit)
Prompt Engineering
(System Prompt + User Query + Context)
Large Language Model
(GPT, Gemini, Claude, Llama, etc.)
Post Processing Layer
(Citation, Formatting, Hallucination Check)
Final Response

Work Flow:


User
Paste Job Description
Chatbot API (FastAPI)
Preprocess Job Description
Retrieve Relevant Information
from Knowledge Base (RAG)
Generate Proposal/Cover Letter using LLM
Format Proposal/Cover Letter


Generate PDF
Return PDF to User